Human-Centered Neuroengineering LAB

C Programming (1)

Week 2B. Recall last week

Department of Embedded System Engineering
Incheon National University

Instructor: Kyungho Won
kyungho.won@inu.ac.kr
Human-Centered Neuroengineering (HCN) Lab

INU 인천대학교 INCHEON NATIONAL UNIVERSITY

* These lecture slides are for class use only. Unauthorized distribution or modification is prohibited.

Recall last week:
Data types & Practice

Recall: Week 2A

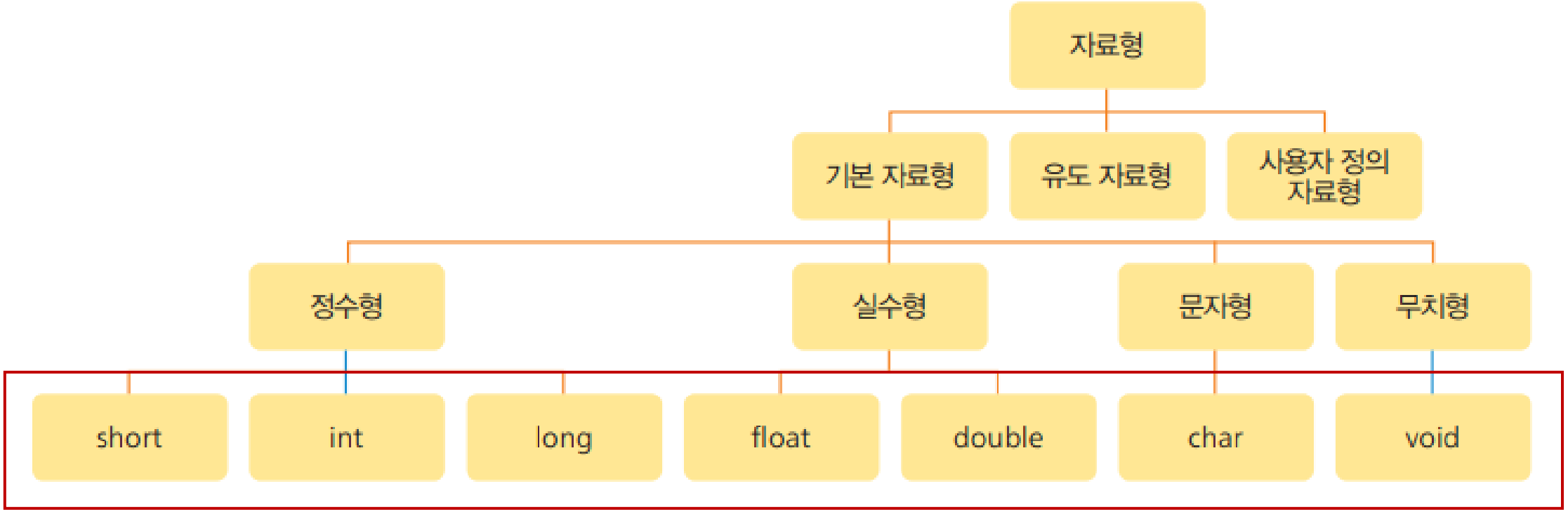
- What are data types?
- Integer types: int, short, long, long long
- Signed and unsigned integers
- Floating-point types: float, double
- Character type: char and ASCII
- Variables: declaration, initialization, naming

Recap: What is a Data Type?

- Data type = tells the compiler:
- • What kind of value is stored
- • How much memory to reserve
- • What operations are valid

- C is statically typed:
- → every variable must have a declared type
- → type cannot change after declaration

Recap: C Data Type Hierarchy



Basic types: integer · floating-point · character · void

Recap: Overview of Basic Types

- Integer: int, short, long, long long
- Unsigned: unsigned int, unsigned char, ...
- Floating-pt: float, double, long double
- Character: char
- Void: void (no value — used in functions)

- Each type has a different size and range
- → choosing the right type matters in embedded systems!

Recap: int: Sizes and Limits

limits.h defines INT_MIN, INT_MAX, SHRT_MIN, LONG_MAX, etc.

```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    printf("int size:  %d bytes\n", (int)sizeof(int));
    printf("int min:   %d\n", INT_MIN);
    printf("int max:   %d\n", INT_MAX);
    return 0;
}
```

Recap: Signed vs Unsigned

- signed → negative AND positive values (default)
- unsigned → zero and positive only

- signed int: −2,147,483,648 to +2,147,483,647
- unsigned int: 0 to +4,294,967,295

- Use unsigned for:
- • Counts and sizes (can't be negative)
- • Bit masks in embedded systems
- • Hardware register values

Recap: Signed vs Unsigned Example

```c
#include <stdio.h>

int main(void)
{
    unsigned int u = 4294967295U;   /* max unsigned */
    int          s = -1;

    printf("unsigned: %u\n", u);    /* 4294967295 */
    printf("signed:   %d\n", s);    /* -1          */

    /* Mixing causes subtle bugs! */
    if (s < u)          /* signed -1 > unsigned max! */
        printf("s < u\n");
    else
        printf("s >= u  (unexpected!)\n");
    return 0;
}
```

Recap: Integer Overflow

- Overflow: value exceeds the type's range

  → wraps around silently (no crash, no warning!)

```
int x = 2147483647;    /* INT_MAX */
x = x + 1;             /* becomes -2147483648! */
```

- Critical in embedded systems:
- • Sensor counter wraps: 255 + 1 = 0 for uint8_t
- • Timestamps can overflow

  → always choose a type with enough range

8 bit

Recap: Integer Literals: Decimal, Octal, Hex

%X prints in uppercase hex, %x in lowercase

```c
#include <stdio.h>

int main(void)
{
    int dec = 255;
    int oct = 0377;    /* octal   (prefix 0)  */
    int hex = 0xFF;    /* hex     (prefix 0x) */

    /* All three are the same value */
    printf("dec = %d\n", dec);  /* 255 */
    printf("oct = %d\n", oct);  /* 255 */
    printf("hex = %d\n", hex);  /* 255 */

    /* Print IN hex or octal */
    printf("in hex: %X\n", dec);  /* FF  */
    printf("in oct: %o\n", dec);  /* 377 */
    return 0;
}
```

Recap: Practice 1: Data Type Sizes

- Write a program that prints the size (bytes) of:
- short, int, long, long long, float, double, char

- Use sizeof() for each type
- Format: "int: 4 bytes" (use field width for alignment)

Recap: Practice 1 — Solution

```c
#include <stdio.h>

int main(void)
{
    printf("char:      %d bytes\n", (int)sizeof(char));
    printf("short:     %d bytes\n", (int)sizeof(short));
    printf("int:       %d bytes\n", (int)sizeof(int));
    printf("long:      %d bytes\n", (int)sizeof(long));
    printf("long long: %d bytes\n", (int)sizeof(long long));
    printf("float:     %d bytes\n", (int)sizeof(float));
    printf("double:    %d bytes\n", (int)sizeof(double));
    return 0;
}
```

Recap: Practice 2: Temperature Converter

- Read a Celsius temperature from the user
- Convert to Fahrenheit: $F = (C \times 9 / 5) + 32$
- Print: "25.0C = 77.0F"

⚠ Watch out for integer division in C * 9 / 5!

- Use 9.0 and 5.0 to force floating-point division

Recap: Practice 2 — Solution

Use 9.0 and 5.0 (not 9 and 5) to avoid integer division!

```
#define _CRT_SECURE_NO_WARNINGS // always above #include <stdio.h>
#include <stdio.h>

int main(void)
{
double celsius, fahrenheit;


printf("Celsius: ");
scanf("%lf", &celsius);


// scanf가 1개의 값을 성공적으로 읽었는지 검사
//if (scanf("%lf", &celsius) != 1) {
// printf("입력 오류: 올바른 숫자가 아닙니다.₩n");
// return 1; // 프로그램 비정상 종료
//}


// or
// scanf_s("%lf", &celsius);


fahrenheit = (celsius * 9.0 / 5.0) + 32.0;
printf("%.1fC = %.1fF₩n", celsius, fahrenheit);
return 0;
}
```

C6031	반환 값이 무시되었습니다. 'scanf'.
C4996	'scanf': This function or variable may be unsafe. Consider using scanf_s instead. To disable deprecation, use _CRT_SECURE_NO_WARNINGS. See online help for details.

Recap: Floating-Point Comparison Trap

- Floating-point numbers are approximations:
- 0.1 + 0.2 == 0.3 → FALSE in C!
- (actual stored value: 0.30000000000000004)

- Never use == to compare floats:

❌ if (x == 0.1) /* unreliable */

✅ if (fabs(x - 0.1) < 1e-9) /* correct */

- #include <math.h> for fabs()

Recap: The char Type

- char = 1 byte, stores a single character
- char letter = 'A';
- char digit = '5';
- char space = ' ';
- char newline = '₩n';

- Single quotes for characters: 'A' ← char
- Double quotes for strings: "Hello" ← string (Week 7)

- Internally stored as its ASCII integer code

Recap: char and ASCII Values

'A'=65, 'a'=97 → difference is 32 (case conversion trick)

```
#include <stdio.h>

int main(void)
{
    char c = 'A';

    printf("char:      %c\n", c);        /* A  */
    printf("ASCII:     %d\n", c);        /* 65 */
    printf("Next:      %c\n", c + 1);    /* B  */
    printf("Lowercase: %c\n", c + 32);   /* a  */

    /* Key ASCII values to remember: */
    /* 'A'=65  'a'=97  '0'=48  ' '=32 */
    return 0;
}
```

Recap: char Arithmetic Example

We'll learn for loops in later weeks — notice how char arithmetic works

```
#include <stdio.h>

int main(void)
{
    char c;

    /* Print A to Z */
    for (c = 'A'; c <= 'Z'; c++)
        printf("%c ", c);
    printf("\n");

    /* Check if digit */
    char ch = '7';
    if (ch >= '0' && ch <= '9')
        printf("%c is a digit, value = %d\n", ch, ch - '0');

    return 0;
}
```

Recap: Practice 3: Character Info

- Read a single character from the user
- Print:
- • The character itself
- • Its ASCII code
- • Whether it is a digit, uppercase, or lowercase

- Hint: use ranges like `c >= 'A' && c <= 'Z'`

Recap: Practice 3 — Solution

```c
#include <stdio.h>

int main(void)
{
    char c;
    printf("Enter a character: ");
    scanf(" %c", &c);

    printf("Character: %c\n", c);
    printf("ASCII:     %d\n", c);

    if (c >= '0' && c <= '9')
        printf("Type: digit\n");
    else if (c >= 'A' && c <= 'Z')
        printf("Type: uppercase\n");
    else if (c >= 'a' && c <= 'z')
        printf("Type: lowercase\n");
    else
        printf("Type: other\n");
    return 0;
}
```

Human-Centered Neuroengineering LAB

C Programming (1)

Week 2B. Variables and Data Types - continue

Department of Embedded System Engineering
Incheon National University

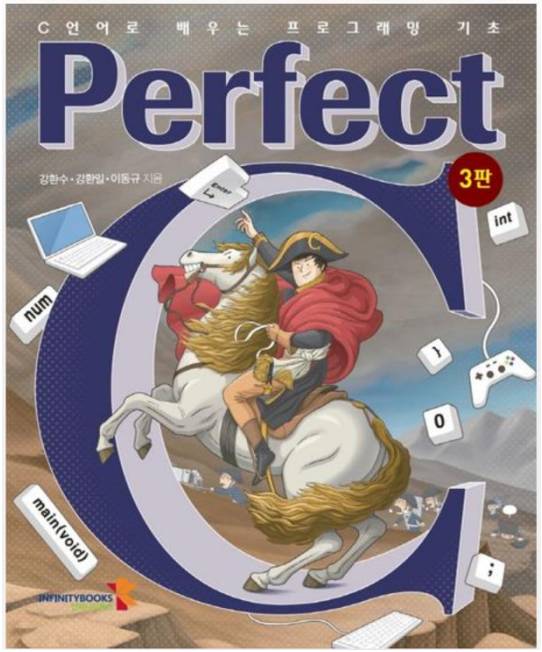
Instructor: Kyungho Won
kyungho.won@inu.ac.kr
Human-Centered Neuroengineering (HCN) Lab

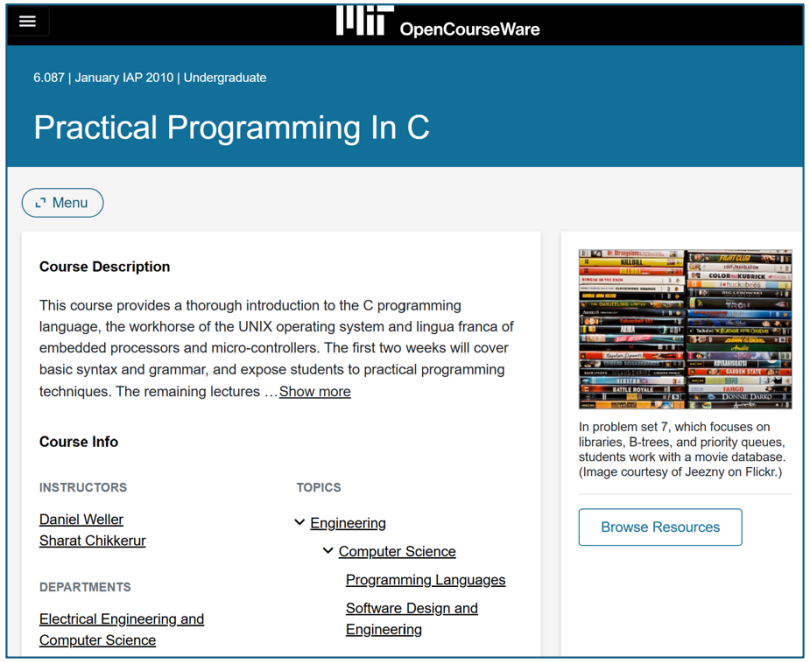
INU 인천대학교 INCHEON NATIONAL UNIVERSITY

* These lecture slides are for class use only. Unauthorized distribution or modification is prohibited.

Class Materials

- Lecture slides
  - Slides (PDF) will be distributed 1-hour earlier
- Perfect C 3판
  - 강환수, 강환일 이동규 저
  - 인피니티북스
- Practical Programming in C
  - Daniel Weller, Sharat Chikkerur
  - MIT OpenCouseWare





Course Syllabus (tentative)

1st week	- Introduction, Hello World, Compiling & Debugging	
2nd week	- Variables and Data Types	3.12 개교기념일 온라인 보강영상
3rd week	- Operators, Expressions, Preprocessor & I/O	
4th week	- Control Flow – Conditions (if, switch)	
5th week	- Control Flow – Loops (for, while, do-while)	
6th week	- Arrays	
7th week	- Functions	
8th week	Midterm	

Variables

Variable Declaration

변수선언은 컴파일러에게 이 소스에서 사용할 변수의 이름과 분류인 자료형을 알려주며, 컴파일러는 실제 변수선언 문장에 맞는 저장 영역을 메모리에 확보한다.

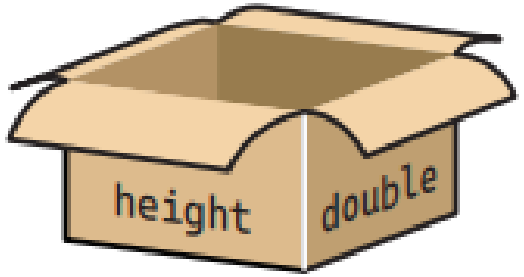
한 학과의 학생을 학번으로 구별하듯이 변수도 고유한 변수 이름으로 변수를 서로 구별할 수 있어야 한다. 일반적으로 한 함수에서 변수의 이름은 반드시 서로 구별되어야 한다.

변수선언문

자료형 변수이름

```
double height;
```

자료형 double, 변수이름 height로 실수를 저장할 수 있는 공간을 확보



type name; → reserves a named memory box of the correct size

Variable Declaration Syntax

- Syntax: type name;
- type name = initial_value;

```
int    age    = 20;
double salary = 3500.50;
char   grade  = 'B';
float  temp   = 36.5f;
```

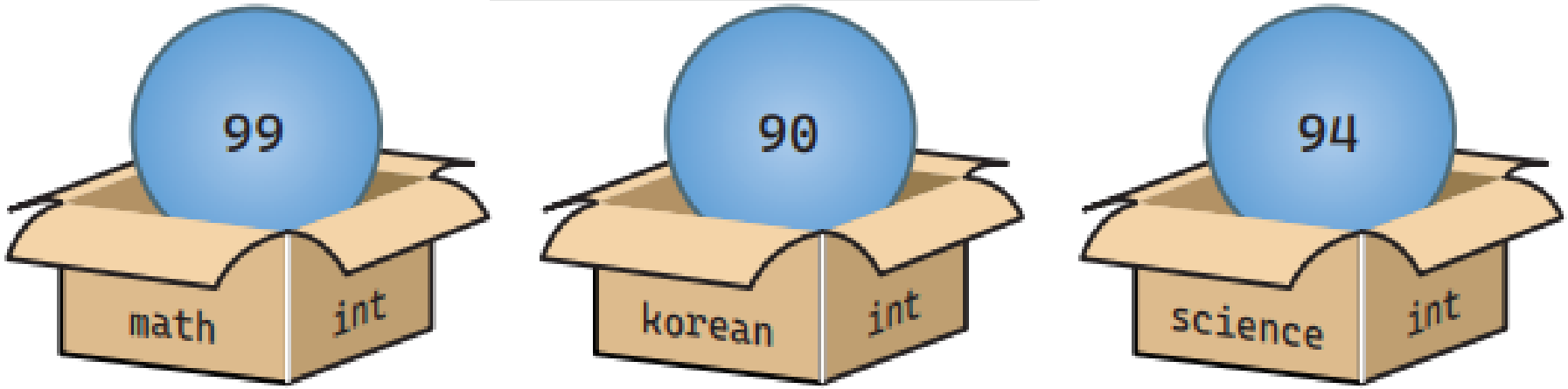
- Always initialize — uninitialized variables hold garbage!

```
int x;  printf("%d", x);  ← unpredictable output
```

Multiple Variables in Memory

```
int math = 99, korean = 90, science = 94;
```

```
int math = 99;
int korean = 90;
int science = 94;
```



Each variable gets its own named memory location

Declaring Multiple Variables

```
/* Separate lines — recommended for readability */
int math    = 99;
int korean  = 90;
int science = 94;

/* Same line — compact */
int math = 99, korean = 90, science = 94;

/* Declare first, assign later */
int a, b, c;
a = 10;
b = 20;
c = a + b;
```

Variable Naming Rules

✅ Valid names:

- • Letters (a-z, A-Z), digits (0-9), underscore _
- • Must start with a letter or _
- • Case-sensitive: count ≠ Count ≠ COUNT

❌ Invalid names:

- • 2fast (starts with digit)
- • my-var (hyphen not allowed)
- • int (reserved keyword)
- • my var (no spaces)

Naming Best Practices

- Use descriptive names:

  ❌ int x, y, z;

  ✅ int width, height, area;

- Multi-word names: use snake_case

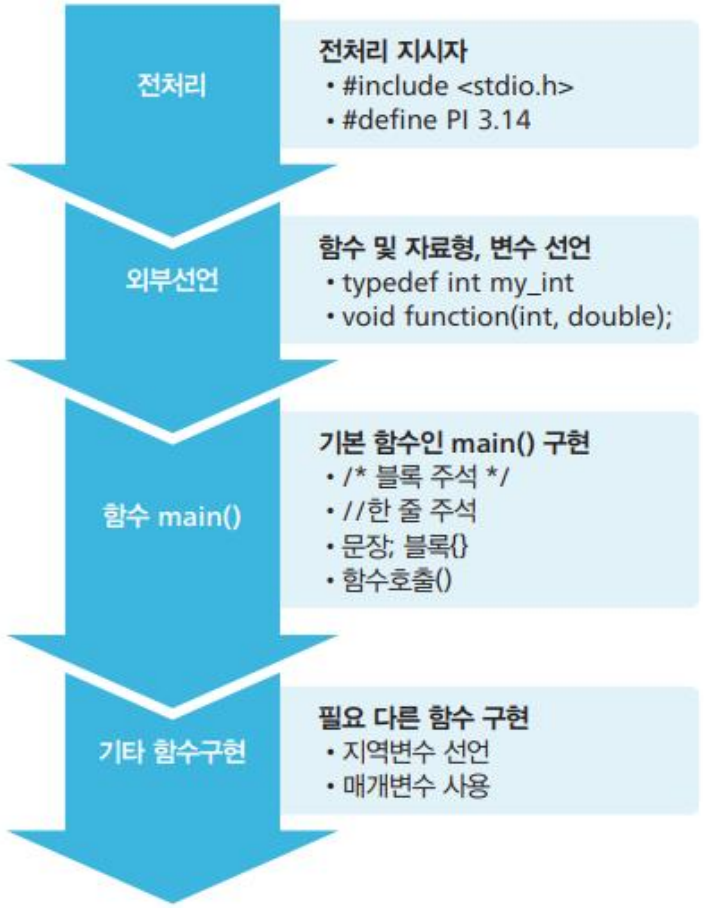
  ✅ student_count, max_speed, led_status

- Constants: ALL_CAPS

  ✅ MAX_SIZE, BAUD_RATE, PI

- Short names ok only for loop counters: i, j, k

Where Variables Live in a Program



Global (before main) vs Local (inside functions) — we'll cover scope in detail in later weeks

Local vs Global Variables (Preview)

Local variable: declared inside a function

- • Only exists while that function runs
- • Cannot be accessed from other functions

Global variable: declared outside all functions

- • Exists for the entire program lifetime
- • Accessible from any function

Best practice: prefer local variables

- → global variables cause hard-to-find bugs
- (Covered in depth in Week 6: Functions)

Constants

Why Use Constants?

- Magic numbers make code hard to understand and change:

❌ area = 3.14159 * r * r;

✅ area = PI * r * r;

❌ if (temp > 85) /* what is 85? */

✅ if (temp > MAX_TEMP) /* clearly a limit */

- Two ways to define constants in C:

1. #define (preprocessor macro)
2. const (typed constant variable)

#define vs const

Prefer const — it has a type, respects scope, and debuggers can see it

```
/* Method 1: preprocessor macro (no type, no semicolon) */
#define PI        3.14159
#define MAX_SIZE  100
#define GREETING  "Hello"

/* Method 2: const variable (has type, has semicolon) */
const double PI       = 3.14159;
const int    MAX_SIZE = 100;

/* Usage */
double area = PI * r * r;
int    buf[MAX_SIZE];
```

const in Embedded Systems

- const is especially useful in embedded systems:

```
const int  LED_PIN   = 13;
const int  BAUD_RATE = 9600;
const int  MAX_TEMP  = 85;
const char VERSION[] = "v1.0";
```

- Compiler may store const data in ROM (flash memory)

  → saves precious RAM on microcontrollers

  → data is read-only (can't be modified accidentally)

Type Conversion

Implicit (Automatic) Conversion

- Compiler converts types automatically in expressions:

```
int    i = 5;
double d = i;   /* int → double: safe, no data loss */
```

- Widening (safe): smaller → larger type
- char → int → long → float → double

- Narrowing (risky): larger → smaller type
- double → int truncates decimal part
- int → char may lose high bits

Implicit Conversion Examples

Narrowing conversions compile but may lose data — be careful

```c
#include <stdio.h>

int main(void)
{
    int    i = 7;
    double d = 2.5;
    double result;

    result = i + d;        /* 7 → 7.0, then 7.0 + 2.5 = 9.5 */
    printf("%.1f\n", result);

    int truncated = 9.99;  /* 9.99 → 9: decimal part LOST! */
    printf("%d\n", truncated);

    char c = 300;          /* 300 overflows char: undefined! */
    printf("%d\n", c);
    return 0;
}
```

The Integer Division Trap

- When both operands are int → result is int (truncated):

```
int a = 7, b = 2;
double result = a / b;    /* result = 3.0, NOT 3.5! */
                           /* division happens FIRST */
```

Fix: cast one operand before division:

```
double result = (double)a / b;   /* = 3.5 */
```

- One of the most common beginner bugs in C!

Explicit Casting

Syntax: (type) expression — cast BEFORE the operation, not after

```
#include <stdio.h>

int main(void)
{
    int a = 7, b = 2;
    double d = 9.99;

    printf("int div:      %d\n",   a / b);            /* 3   */
    printf("cast first:   %.1f\n", (double)a / b);    /* 3.5 */
    printf("cast other:   %.1f\n", a / (double)b);    /* 3.5 */
    printf("cast result:  %.1f\n", (double)(a / b)); /* 3.0 — wrong! */

    printf("truncate:     %d\n",   (int)d);            /* 9   */
    return 0;
}
```

sizeof() Operator

sizeof() works on types AND variables — very useful in embedded systems

```c
#include <stdio.h>

int main(void)
{
    int    x = 42;
    double d = 3.14;

    /* Size of types */
    printf("char:   %d bytes\n", (int)sizeof(char));
    printf("int:    %d bytes\n", (int)sizeof(int));
    printf("double: %d bytes\n", (int)sizeof(double));

    /* Size of variables */
    printf("x:      %d bytes\n", (int)sizeof(x));
    printf("d:      %d bytes\n", (int)sizeof(d));
    return 0;
}
```

Type Conversion Summary

Implicit (automatic):

- • Safe: smaller → larger (char→int, int→double)
- • Risky: larger → smaller (may lose data)

Explicit (cast): (type) expression

- • Forces conversion you intend
- • Silences compiler warnings

Integer division:

- • Both int → result is int
- • Cast one to double BEFORE division

Practice 5: Casting Challenge

- Declare: int a = 7; int b = 2;
- Predict the result BEFORE running, then verify:
- • a / b
- • (double)a / b
- • a / (double)b
- • (double)(a / b)

- Explain why the 4th result differs from 2nd and 3rd

Practice 5 — Solution

4th: int division (3) happens FIRST, then cast to 3.0 — too late!

```
#include <stdio.h>

int main(void)
{
    int a = 7, b = 2;

    printf("a / b             = %d\n",   a / b);          /* 3   */
    printf("(double)a / b     = %.1f\n", (double)a / b);  /* 3.5 */
    printf("a / (double)b     = %.1f\n", a / (double)b);  /* 3.5 */
    printf("(double)(a / b)   = %.1f\n", (double)(a/b));  /* 3.0 */
    return 0;
}
```

Practice 6: Hex and Overflow

Part A: Print 255 in decimal, octal, and hex

- using %d, %o, %X format specifiers

Part B: Demonstrate overflow:

- char c = 127; /* max signed char */
- c = c + 1; /* what happens? */
- printf("%d₩n", c);

Predict Part B output, then run and explain the result

Practice 6 — Solution

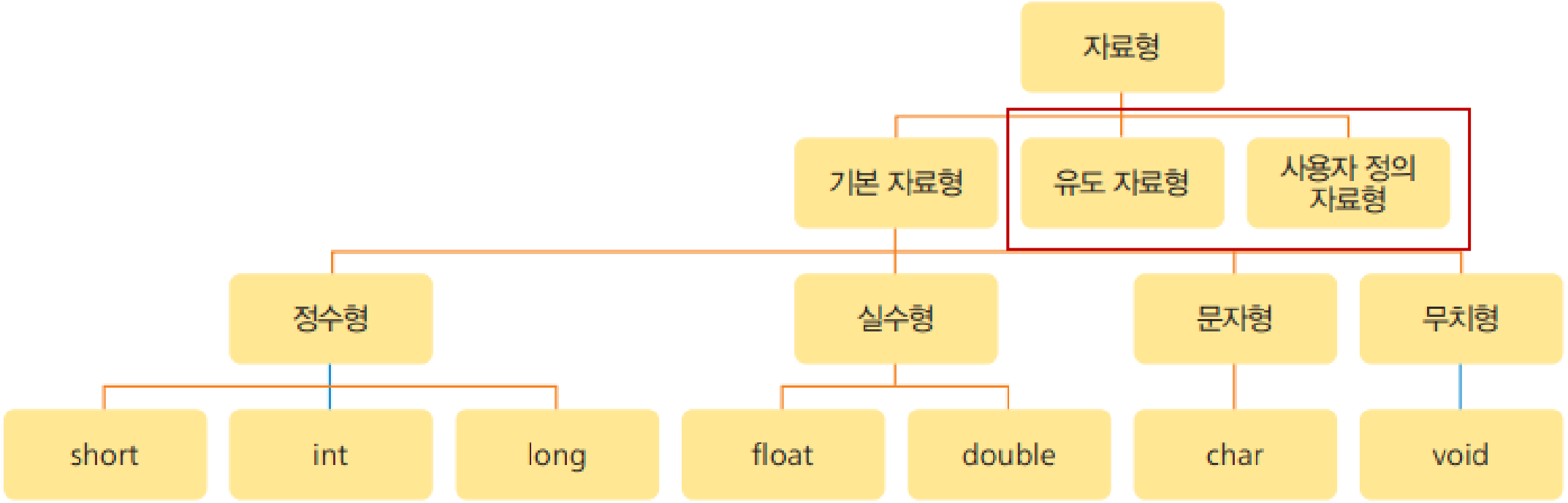
Part B: 127 + 1 = -128 because signed char wraps around!

```c
#include <stdio.h>

int main(void)
{
    /* Part A */
    int n = 255;
    printf("dec: %d\n", n);   /* 255 */
    printf("oct: %o\n", n);   /* 377 */
    printf("hex: %X\n", n);   /* FF  */

    /* Part B: char overflow */
    char c = 127;              /* CHAR_MAX */
    c = c + 1;                 /* wraps to -128 */
    printf("overflow: %d\n", c);  /* -128 */
    return 0;
}
```

Recap: C Data Type Hierarchy



Basic types: integer · floating-point · character · void

Derived Types (유도 자료형)

- **Built by combining or extending basic types**
- **Array (배열)**: store multiple values of same type

```
int scores[5] = {90, 85, 77, 92, 88};
```

- **Pointer (포인터)**: stores address of another variable

```
int *p = &x;   /* p holds address of x */
```

- **Function (함수)**: defined by `return` type + parameters

```
int add(int a, int b) { return a + b; }
```

→ These will be covered in detail in later weeks!

User-Defined Types (사용자 정의 자료형)

- **Programmer defines new types to model real-world data**
- **struct (구조체)** — group different types together
- **union (공용체)** — members share the same memory
- **enum (열거형)** — named integer constants
- **typedef** — alias for existing types

→ Improves code readability

→ Enables complex data structure modeling

→ Essential for real-world C programming

struct — Grouping Different Types

- **struct bundles different types into one unit**

```
struct Student {
    int id;  char name[20];  double gpa;
};
```

```
struct Student s1;
s1.id = 2025001;   s1.gpa = 3.85;
```

→ Each member has own memory | Total size ≥ sum of members

union — Shared Memory

- **All members share the same memory — only one valid at a time!**

```
union Data {
    int i;  float f;  char c;
};
```

```
union Data d;
d.i = 42;       /* now i is valid */
d.f = 3.14f;    /* f valid, i overwritten! */
```

→ Size = largest member (4 bytes, not 4+4+1=9)

struct vs union: Memory Layout

- **struct: each member gets its OWN memory**

  | `int` i (4B) | `float` f (4B) | `char` c (1B) | → 12 bytes*

- **union: all members SHARE the same memory**

  | `int` i / `float` f / `char` c | → 4 bytes

`sizeof(struct)` → 12 (with padding*) | `sizeof(union)` → 4

* `struct` may add padding `for` memory alignment

- **struct → grouped data | union → memory-saving alternatives**

enum — Named Constants

- **enum defines a set of named integer constants**

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }; /* MON=0, TUE=1, WED=2, ... SUN=6 */

enum Day today = WED;

printf("WED = %d\n", today);   /* prints 2 */
```

- **Custom values:**

```
enum Color { RED = 1, GREEN = 2, BLUE = 4 };
```

→ Much more readable than magic numbers!

```
 if (today == WED)  vs  if (today == 2)
```

typedef — Type Aliases

- **typedef creates an alias (별명) for existing types**

```
typedef unsigned int  uint;

typedef unsigned char byte;

uint count = 100;   byte flag = 0xFF;
```

- **Commonly used with struct:**

```
typedef struct { int x; int y; } Point;

Point p1 = {10, 20};  /* no need to write "struct" */
```

User Input with scanf()

Getting User Input

- scanf() reads data typed by the user:
- Syntax: scanf("format", &variable);

- Key rule: & (address-of) before the variable name!

✅ scanf("%d", &age);

❌ scanf("%d", age); ← crashes!

- & means 'address of' — tells scanf WHERE to store the value

scanf() Examples

scanf %lf for double, but printf %f — easy to mix up!

```
#include <stdio.h>

int main(void)
{
    int    age;
    double height;
    char   grade;

    printf("Age: ");
    scanf("%d", &age);

    printf("Height (m): ");
    scanf("%lf", &height);    /* %lf for double! */

    printf("Grade: ");
    scanf(" %c", &grade);     /* space before %c */

    printf("age=%d  height=%.2f  grade=%c\n",
           age, height, grade);
    return 0;
}
```

scanf() Tips

- Format specifiers for scanf:

```
%d   → int
%f   → float
%lf  → double   (different from printf!)
%c   → char     (add space before: " %c")
%ld  → long
```

- Common mistakes:
  - ❌ Forgetting & → program crashes
  - ❌ Using %f for double → wrong value stored

Practice 3: Circle Calculator

- Define: #define PI 3.14159
- Read radius r from the user (double)

- Calculate and print:
- • Circumference: 2 × PI × r
- • Area: PI × r × r

- Print with 2 decimal places

Practice 3 — Solution

```
#define _CRT_SECURE_NO_WARNINGS // always above #include <stdio.h>

#include <stdio.h>
#define PI 3.14159

int main(void)
{
    double r;
    printf("Radius: ");
    scanf("%lf", &r);

    printf("Circumference: %.2f\n", 2.0 * PI * r);
    printf("Area:          %.2f\n", PI * r * r);
    return 0;
}
```

Common Mistakes This Week

- ❌ Uninitialized variables → garbage values
- ❌ Integer division: 7/2 = 3, not 3.5
- ❌ Casting after division: (double)(7/2) = 3.0
- ❌ Using == to compare floats
- ❌ 'A' vs "A" — char uses single quotes
- ❌ Forgetting & in scanf
- ❌ Using %f instead of %lf in scanf for double

Study Tips

- Data types are the foundation of C — learn sizes and ranges
- Always initialize variables when declaring them
- When mixing types, think: widening or narrowing?
- Always test the boundary cases (min, max values)
- Use sizeof() when you're unsure about a type's size

Week 2 Summary

- int/short/long/long long: integers with different ranges
- signed vs unsigned: unsigned doubles positive range
- float/double: decimals (prefer double for precision)
- char: 1-byte character, stored as ASCII integer
- Overflow is silent — choose types carefully!
- const / #define: two ways to define named constants
- Casting: (type)expr — cast BEFORE operations
- scanf(): read input, always use &, use %lf for double

Next Week: Operators & I/O

- Deep dive into operators:
- • Arithmetic: + - * / % and precedence
- • Relational: == != < > <= >=
- • Logical: && || ! and short-circuit evaluation
- • Assignment: += -= *= /= %=
- • Increment/Decrement: ++ --
- printf() format specifiers in depth (width, precision)
- scanf() in depth with multiple inputs